

Java & OOP — IBM MCQ Master Notes

Topic-wise revision notes covering the supplied IBM Java syllabus. Focus: definitions, rules, exceptions, output traps, Java 8+, JVM, and commonly tested MCQ facts.

1. Introduction to Java and OOPS

Core Java facts

Java is class-based, object-oriented, statically typed, strongly typed, garbage-collected, and designed for platform independence through bytecode and the JVM. Source code (.java) is compiled by javac into bytecode (.class). JVM executes bytecode.

OOP pillars

Encapsulation = bundling state + behavior and controlling access. Inheritance = acquiring/reusing behavior from a superclass. Polymorphism = one interface/reference, many implementations; overloading is compile-time, overriding is runtime. Abstraction = expose essential behavior while hiding implementation.

JDK/JRE/JVM

JDK = tools + JRE. JRE = JVM + libraries needed to run Java applications. JVM loads, verifies and executes bytecode. Modern JDK distributions may not ship a separate JRE product, but the conceptual relationship remains useful for MCQs.

MCQ traps

Java does not support multiple inheritance of classes. It supports multiple interface inheritance/implementation. Java is pass-by-value; object references are passed by value. Java has no pointer arithmetic. main is traditionally public static void main(String[] args).

2. Software Installation and First Java Program

Compilation/execution

`javac Hello.java` → `Hello.class`; `java Hello` runs the class. The public class name must match the filename. Java is case-sensitive.

Program anatomy

class defines a type. main is the conventional application entry point. `System.out` is a `PrintStream`; `println` prints and terminates the line.

MCQ traps

A .java file can contain multiple top-level classes, but at most one public top-level class, and its name must match the file. A class can have no main method and still compile; it simply cannot be launched using the standard java launcher as an application.

3. Static Members and Execution Control Flow

Static

static members belong to the class, not individual objects. A static field has one class-level copy. Static methods can be called through the class name.

Initialization order

When a class is initialized: static fields and static initializer blocks execute in textual order. Superclass initialization occurs before subclass initialization. Then instance fields/initializers and constructor execute when an object is created.

Static method restrictions

A static method cannot directly access an instance field/method or use `this`/`super` because no particular object is implied. It can access static members directly.

Trap

Static methods are hidden, not overridden. A static method selected through a reference is resolved based on the reference/class context, not dynamic dispatch.

4. Non-Static Members and Execution Control Flow

Object members

Each object normally has its own instance fields. Instance methods operate on a particular object.

Construction order

For new Child(): memory/default values → superclass instance initialization/constructor chain → child instance field initializers and instance initializer blocks in textual order → child constructor body.

Default values

Numeric fields default to 0/0.0, boolean false, char '      ', and references null. Local variables do not receive automatic default values; they must be definitely assigned before use.

5. Datatypes, Literals, Variables, Conversion, Casting & Promotion

Primitive types

byte 8-bit, short 16-bit, int 32-bit, long 64-bit; float 32-bit, double 64-bit; char 16-bit unsigned UTF-16 code unit; boolean represents true/false.

Widening

Generally safe implicit conversion: byte→short→int→long→float→double; char→int→long→float→double. Note that widening to float/double can lose integer precision even though it is permitted.

Narrowing

Requires explicit cast, e.g. int i=10; byte b=(byte)i. Narrowing can overflow/wrap for integral types.

Numeric promotion

byte/short/char operands are generally promoted to int in arithmetic. If either operand is long, result is long; float → float; double → double.

Literals

Integer literals are int by default; suffix L/l makes long. Floating literals are double by default; F/f makes float. 0b binary, 0 octal, 0x hexadecimal. Underscores may improve readability but cannot be placed arbitrarily.

Trap

char + char gives int. 'A' + 1 is an int. String concatenation with + is left-to-right and once a String is involved, later operands are converted to strings.

6. Wrapper Classes, Autoboxing & Unboxing

Wrappers

byte→Byte, short→Short, int→Integer, long→Long, float→Float, double→Double, char→Character, boolean→Boolean. Wrappers are objects and immutable.

Boxing

Integer x = 10; automatically converts int to Integer. Unboxing converts Integer to int.

Null trap

Integer x=null; int y=x; throws NullPointerException due to unboxing null.

Caching

Common wrapper caches exist for some values; Integer autoboxing commonly reuses cached instances for -128..127, but code should never rely on identity (==) for wrapper value comparison. Use equals() or primitive comparison.

7. Operators and Assignments

Operators

Arithmetic + - * / %; relational < <= > >=; equality == !=; logical && || !; bitwise & | ^ ~; shifts << >> >>>; conditional ?; assignment = and compound assignments += etc.

Short-circuit

&& evaluates right side only if needed; || evaluates right side only if needed. & and | on booleans evaluate both operands.

Assignment conversion

Compound assignment such as x += y includes an implicit cast after the operation (subject to type rules). x = x + y may fail where x += y compiles.

Equality trap

== compares primitive values, but references with == are compared by identity/reference. equals() is for logical value equality when overridden.

8. Flow Control Statements

Selection

if/else, switch. switch supports byte/short/int/char, String, enum and corresponding wrapper types in modern Java. long, float and double are not valid switch selector types.

Loops

for, enhanced for, while, do-while. break exits loop/switch; continue skips to next iteration. Labels can target break/continue.

Switch traps

case labels must be compatible constant expressions (traditional switch rules). Duplicate case labels are compile-time errors. Missing break can cause fall-through in traditional switch.

Modern switch note

Java 14 introduced switch expressions as a standard feature, with yield used for multi-statement expression blocks.

9. Access Modifiers

Visibility

public: broadly accessible. protected: same package + subclasses outside package (with qualified-access nuances). package-private/default: same package only. private: declaring class only.

Top-level types

Top-level classes/interfaces cannot be private or protected; top-level class can be public or package-private.

Trap

protected is not simply 'subclasses only'; same-package classes also have access.

10. Packages

Purpose

Packages organize classes, prevent naming conflicts, and provide access control.

Syntax

package declaration normally comes first (apart from comments/annotations). import makes names available without qualification; it does not copy classes.

Static import

import static pkg.Type.member; allows direct use of static members.

Trap

java.lang is implicitly imported. Classes in the same package do not need import. Subpackages are separate packages.

11. Event Management Use Case

Concept

An event-management application is useful for applying OOP: Event, User/Organizer, Venue, Ticket/Registration and Payment can be modeled as classes.

OOP mapping

Encapsulation: private fields + methods. Inheritance: OnlineEvent extends Event. Abstraction: abstract Event or interface Payment. Polymorphism: Payment p = new CardPayment(); p.pay().

MCQ focus

Prefer composition where an object 'has-a' component and inheritance where it genuinely represents an 'is-a' relationship. Interfaces are ideal for interchangeable behaviors.

12. Inheritance

Basics

extends creates class inheritance. A subclass inherits accessible members; constructors are not inherited. private members are not directly accessible in subclasses.

super

super() invokes superclass constructor; super.method() can invoke superclass behavior. If a constructor has no explicit first constructor invocation, compiler inserts super() when applicable.

final

final class cannot be extended; final method cannot be overridden; final variable can be assigned once.

Multiple inheritance

A class can extend only one class. It can implement multiple interfaces.

13. Abstraction

Abstract class

May contain abstract and concrete methods, fields, constructors and static methods. Cannot be instantiated directly.

Interface

Methods can be abstract, default, static, and private depending on Java version; fields are implicitly public static final. An interface itself is not instantiated.

Trap

An abstract class may have zero abstract methods. A class implementing an interface must implement its abstract methods unless the class is abstract.

14. Polymorphism

Overloading

Same method name, different parameter list; resolved at compile time. Return type alone cannot distinguish overloaded methods.

Overriding

Subclass supplies a compatible implementation of an inherited instance method; selected at runtime for virtual dispatch.

Rules

Cannot reduce visibility while overriding. Cannot override final/static/private methods (static methods are hidden; private methods are not inherited). Covariant return types are allowed for overriding.

Trap

Fields are not polymorphic like overridden methods; field access is based on the reference type.

15. Encapsulation

Definition

Hide internal state and expose controlled operations. Common implementation: private fields + public/protected methods.

Benefits

Validation, maintainability, reduced coupling, invariant protection.

Trap

Encapsulation is not synonymous with 'all fields private'; it is about controlled access to representation/behavior.

16. Exception Handling and Assertions

Hierarchy

Throwable → Error and Exception. RuntimeException is unchecked. Checked exceptions are Exception subclasses excluding RuntimeException subclasses.

Keywords

try, catch, finally, throw, throws. throw actually throws an exception object; throws declares possible exceptions in a method signature.

finally

Normally executes whether exception occurs or not, except abnormal termination such as JVM shutdown. Avoid returning from finally because it can suppress exceptions/override returns.

Checked rules

A checked exception must be caught or declared. Unchecked exceptions need not be declared.

Assertions

assert condition; or assert condition : expression;. Assertions are disabled by default and enabled with JVM flags such as -ea. AssertionError is thrown when an enabled assertion fails.

17. Multithreading

Thread basics

Thread can be created by extending Thread or implementing Runnable; modern code often uses ExecutorService. start() creates a new execution path; directly calling run() does not.

States

NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.

Synchronization

synchronized provides mutual exclusion and memory visibility around monitor-protected regions. Each object has an intrinsic monitor.

wait/notify

Object.wait() releases the monitor and waits; notify/notifyAll signal waiting threads. They must be called while owning the object's monitor.

Trap

sleep() is Thread method and does not release an acquired monitor. wait() is Object method and releases the object's monitor while waiting.

18. Executor Framework Quick Start

Core types

Executor executes tasks; ExecutorService adds lifecycle and Future support; ScheduledExecutorService schedules delayed/repeated tasks.

Methods

execute(Runnable) returns void. submit(...) returns Future. shutdown() stops accepting new tasks but lets submitted tasks finish; shutdownNow() attempts to interrupt running tasks and returns queued tasks.

Callable/Future

Callable can return a value and throw checked exceptions. Future.get() waits for completion unless timed get is used.

Trap

ExecutorService should be shut down when no longer needed, otherwise non-daemon worker threads can keep the application alive.

19. Garbage Collection & Types of Objects

GC

Garbage collection automatically reclaims objects that are no longer strongly reachable. System.gc() is only a request/hint, not a guarantee.

Objects

new creates objects on the heap conceptually; local variables and references may live in stack frames, while JVM implementations may optimize allocation.

Eligibility

An object becomes eligible for GC when unreachable from GC roots, not simply when a reference variable goes out of scope in every conceptual situation.

finalize

finalization is deprecated for removal in modern Java; do not use it for resource management. Prefer try-with-resources and AutoCloseable.

20. Inner Classes

Types

Member inner class, static nested class, local class, anonymous class.

Key distinction

A non-static inner class is associated with an enclosing instance. A static nested class does not require an enclosing object.

Anonymous class

Useful for one-off implementations/overrides; has no explicit class name.

Trap

A non-static inner class can access private members of its enclosing class due to Java's nested-type access rules.

21. String Handling

String

String is immutable and final. String literals are commonly stored in the string pool.

equals vs ==

equals compares content for String; == compares reference identity.

StringBuilder/StringBuffer

StringBuilder is mutable and generally preferred for single-threaded string construction. StringBuffer is synchronized/thread-safe but usually slower.

Important methods

length(), charAt(), substring(), indexOf(), contains(), startsWith(), endsWith(), replace(), split(), trim(), strip() (Java 11+), toLowerCase()/toUpperCase().

Trap

String concatenation in a loop can create many intermediate objects; use StringBuilder for repeated concatenation.

22. IO Streams (File IO)

Streams

Byte streams: InputStream/OutputStream for binary data. Character streams: Reader/Writer for text.

Common classes

FileInputStream/FileOutputStream; FileReader/FileWriter; BufferedInputStream/BufferedOutputStream; BufferedReader/BufferedWriter.

Try-with-resources

Resources implementing AutoCloseable can be declared in try(...); they are closed automatically, including when an exception occurs.

Trap

IOException is checked. close() can itself throw IOException; try-with-resources handles suppression rules for secondary exceptions.

23. Arrays

Basics

Arrays have fixed length and homogeneous element type. `int[] a = new int[5];` length is a field, not a method.

Defaults

Array elements get default values. Multidimensional arrays are arrays of arrays and can be jagged.

Utilities

Arrays.sort(), binarySearch(), copyOf(), equals(), deepEquals(), fill(), toString(), deepToString().

Trap

Array index starts at 0; invalid index causes `ArrayIndexOutOfBoundsException`. Arrays are objects and have a runtime component type.

24. Object Class Methods

Important methods

toString(), equals(Object), hashCode(), getClass(), clone(), wait(), notify(), notifyAll(), and finalize (deprecated).

equals/hashCode

If two objects are equal according to equals(), they must have the same hashCode. Unequal objects may share a hash code.

toString

Default Object.toString() includes class name and hash-related representation; override it for meaningful output.

clone

Object.clone() is protected and performs a shallow field copy when cloning is supported; Cloneable is a marker interface. Prefer copy constructors/factories in new designs.

25. Collections with Generics

Interfaces

List: ordered, duplicates allowed. Set: no duplicate elements. Queue: typically processing order. Map: key-value pairs; Map is not a subtype of Collection.

Common classes

ArrayList, LinkedList, HashSet, LinkedHashSet, TreeSet, HashMap, LinkedHashMap, TreeMap, PriorityQueue.

HashMap

Allows one null key and multiple null values. Ordering is not guaranteed. HashSet is backed by a HashMap conceptually.

TreeMap/TreeSet

Sorted collections based on natural ordering or Comparator.

Generics

Provide compile-time type safety: List<String>. Generics use reference types, not primitives; use List<Integer>, not List<int>.

Wildcards

? extends T is useful for producers/read; ? super T is useful for consumers/write. PECS = Producer Extends, Consumer Super.

Trap

Collection size is size(), arrays use .length, String uses .length().

26. Enums

Basics

enum defines a fixed set of named constants and is a class-like type.

Features

Enums can have fields, constructors, methods and implement interfaces. Enum constructors are not invoked directly with new.

Methods

values(), valueOf(String), name(), ordinal(). ordinal is zero-based and should not be treated as a stable business ID.

Trap

Enum constants are singleton-like instances per enum type/classloader.

27. Java 8 Features

Lambdas

Enable concise behavior values, e.g. $(x) \rightarrow x * 2$. A lambda targets a functional interface.

Functional interfaces

Exactly one abstract method; examples Runnable, Callable, Comparator, Predicate, Function, Consumer, Supplier.
@FunctionalInterface lets compiler enforce the contract.

Streams

Stream pipeline: source $\rightarrow$ intermediate operations (filter/map/sorted) $\rightarrow$ terminal operation (collect/forEach/reduce). Intermediate operations are lazy.

Optional

Container for possibly absent value. Common methods: of, ofNullable, empty, isPresent, orElse, orElseGet, map, flatMap.

Default/static interface methods

Interfaces can provide default implementations and static methods.

Date/time

java.time API: LocalDate, LocalDateTime, Instant, ZonedDateTime, Duration, Period, DateTimeFormatter.

Method references

Class::staticMethod, object::instanceMethod, Class::instanceMethod, Class::new.

28. JVM Architecture

Main areas

Class Loader subsystem loads classes; runtime data areas include heap, JVM stacks, method area/metaspace, PC registers and native method stacks.

Heap

Objects/arrays are allocated here conceptually; shared among JVM threads.

Stack

Each thread has its own JVM stack containing frames for method calls, local variables and operand stack.

Execution engine

Interpreter executes bytecode; JIT compiler compiles frequently executed code to native machine code for performance. Garbage collector manages heap memory.

Class loading

Loading, linking (verification, preparation, resolution as needed), and initialization are major phases.

Trap

JVM specification defines behavior, not one universal physical implementation. Exact memory layout can vary by JVM.

29. Java Annotations

Purpose

Annotations provide metadata to code and can be processed by compiler, tools or runtime.

Built-in examples

@Override, @Deprecated, @SuppressWarnings, @FunctionalInterface, @SafeVarargs.

Meta-annotations

@Target controls applicable locations; @Retention controls lifetime: SOURCE, CLASS, RUNTIME; @Documented; @Inherited; @Repeatable.

Trap

@Inherited affects class-level annotations inherited by subclasses; it does not mean all annotations are inherited.

30. Interview Questions / High-Frequency MCQ Facts

High yield

Difference between == and equals; overloading vs overriding; checked vs unchecked exceptions; final/finally/finalize; static vs instance; String vs StringBuilder; ArrayList vs LinkedList; HashMap vs TreeMap; interface vs abstract class; sleep vs wait; start vs run.

Remember

Constructor has no return type, is not inherited, and cannot be static/final/abstract. A private constructor can exist and is often used for controlled instantiation.

Common trick

Method signature for overloading includes name + parameter types/order, not return type. Exception types in throws do not form part of a method signature for overloading.

Design question

Prefer interfaces for capabilities/contracts and composition for reusable components. Use inheritance when substitutability and an 'is-a' relationship are valid.

31. HTTP Client API

Java 11+

java.net.http.HttpClient, HttpRequest and HttpResponse provide a modern HTTP client API.

Flow

Build HttpClient → HttpRequest → send/sendAsync → HttpResponse. BodyHandlers.string() returns a String body; BodyHandlers.ofByteArray() returns bytes.

Sync vs async

send() blocks until response; sendAsync() returns CompletableFuture.

HTTP versions

HttpClient can be configured with HTTP_1_1 or HTTP_2. Follow redirects and connect timeout can be configured.

Trap

HttpClient was introduced in Java 11, not Java 8.

32. Java 13 and 14 Features

Java 13

Text blocks were introduced as a preview feature in Java 13. Switch expressions remained preview in Java 13.

Java 14

Switch expressions became standard. Text blocks were still preview in Java 14. Helpful NullPointerException messages were introduced.

Switch expression

Can return a value: int x = switch(day) { case 1 -> 10; default -> 0; }; For block-style cases, yield returns the value.

Trap

Do not confuse preview features with standard features. In Java 14, switch expressions are final; text blocks are not yet final.

Rapid MCQ Cheat Sheet

Concept	Must remember
main	public static void main(String[] args)
String	immutable; == identity, equals content
Array	.length
String	.length()
Collection	.size()
Map	not a Collection
static	class-level; hidden, not overridden
final method	cannot be overridden
final class	cannot be extended
private	declaring class only
protected	same package + subclass access rules
overload	compile-time; parameter list differs
override	runtime; compatible return/visibility rules
start()	new thread execution
run()	ordinary method call if called directly
sleep()	does not release monitor
wait()	releases monitor while waiting
execute()	Executor; no Future return
submit()	ExecutorService; returns Future
checked exception	catch or declare
assert	disabled by default; -ea enables
HashMap	unordered; one null key permitted
TreeMap	sorted by key
ArrayList	dynamic array; fast random access
LinkedList	linked structure; also Deque
List	generics require reference type
? extends T	producer/read
? super T	consumer/write
enum ordinal	zero-based; don't use as stable ID
Java 8	lambda, streams, Optional, default methods, java.time
Java 11	HttpClient
Java 14	switch expressions standard

Exam Strategy

For IBM-style MCQs, prioritize: execution order, access modifiers, overloading/overriding, exceptions, collections, String behavior, threads, Java 8 functional programming, JVM areas, and Java 11–14 feature/version questions. When two answers look similar, check whether the question is testing compile-time vs runtime behavior, value vs reference identity, or a checked vs unchecked exception.

Use this PDF as revision notes rather than as a replacement for hands-on code. For output-based MCQs, write down the initialization/dispatch order before choosing an answer.